

Pestaña 1

Aplicación Angular con app.module.ts como módulo principal y 7 componentes principales:

- **Inicio:** Página principal
- **Navbar:** Navegación principal
- **DriverList:** Lista de pilotos con filtros
- **DriverDetail:** Detalle individual de piloto
- **DriverForm:** Formulario CRUD para pilotos
- **CarList:** Lista de automóviles F1
- **CircuitList:** Lista de circuitos

Angular Signals para estado reactivo (en el servicio F1Data):

```
isEditMode = signal<boolean>(false);  
loading = signal<boolean>(false);  
error = signal<string>("");
```

Inyección de dependencias moderna con inject() en componentes:

```
private route = inject(ActivatedRoute);  
private router = inject(Router);  
private f1Data = inject(F1Data);
```

- Tipado estricto con interfaces **TypeScript** (Piloto, Auto, Circuito).
- Componentes con standalone: **false**, declarados en **AppModule**.

Estructura base:

- **interfaces (tipado)**
- **servicioDATA** (lógica y datos)
- **src/app/components/[componentes]** (HTML, CSS y TS)

ROUTING Y NAVEGACIÓN

8 rutas configuradas con navegación fluida:

```
{ path: '', component: InicioComponent, title: 'F1 Universe - Inicio' }  
{ path: 'drivers', component: DriverListComponent, title: 'Pilotos - F1 Universe' }  
{ path: 'driver-detail/:id', component: DriverDetailComponent, title: 'Detalle del Piloto' }  
{ path: 'driver-form/:id', component: DriverFormComponent, title: 'Editar Piloto' }  
{ path: 'driver-form', component: DriverFormComponent, title: 'Nuevo Piloto' }  
{ path: 'cars', component: CarListComponent, title: 'Monoplazas - F1 Universe' }  
{ path: 'circuits', component: CircuitListComponent, title: 'Circuitos - F1 Universe' }  
{ path: '**', redirectTo: '', pathMatch: 'full' }
```

Configuración avanzada del router:

enableTracing: false

scrollPositionRestoration: 'top'

FORMULARIOS Y VALIDACIONES

Formulario reactivo con **FormBuilder**.

Validaciones integradas (**required**, **minLength**, **min**).

Carga de datos en edición con parámetro de ruta:

const id = this.route.snapshot.paramMap.get('id');

Si existe ID: activa edición y rellena el formulario.

SERVICIOS Y HTTPCLIENT

Servicio principal **F1Data**. HttpClient con Angular In-Memory Web API.

BehaviorSubject para estado global reactivo:

private pilotosSubject = new BehaviorSubject<Piloto[]>([]);

public readonly pilotos\$ = this.pilotosSubject.asObservable();

Endpoints:

- **api/pilotos**
- **api/autos**
- **api/circuitos**

CRUD completo para pilotos (**create/read/update/delete**).

- Manejo de errores centralizado con **handleError<T>():**
- Recurso no encontrado (404) y error interno (500)

Limpieza automática del mensaje de error con timeout.

Normalización de rutas de imágenes de pilotos (**/public/... -> /...**) y método **getPilotoDesdeLista(id)**.

SIMULACIÓN DE BACKEND

in-memory-data.service.ts con base de datos en memoria funcional.

Archivo de datos realistas de pilotos, coches y circuitos.

RUTAS CON PARÁMETRO

Obtención de parámetro de ruta:

- `const id = this.route.snapshot.paramMap.get('id');`

Detalle de piloto:

- Muestra información completa del piloto seleccionado.

Acciones CRUD desde detalle:

- Editar (navega con ID)
- Eliminar con confirmación
- Volver a la lista

GUIÓN

0:00 – 0:20 | Introducción

Hola, mi proyecto se llama F1 Universe.

Es una aplicación web en Angular 21 para consultar pilotos, coches y circuitos.

Los pilotos sí se pueden gestionar con CRUD; coches y circuitos son solo consulta.

0:20 – 1:00 | Estructura del proyecto

Abrir en VS Code: app

Mostrar: carpeta components

Aquí están todos los componentes organizados por carpetas, cada uno con HTML, CSS y TS.

Abrir: piloto.interface.ts

Aquí defino el tipado estricto del piloto. Lo mismo ocurre con autos y circuitos.

Abrir: app.module.ts

Este es el módulo principal, donde declaro componentes e importo HttpClientModule, ReactiveFormsModule, RouterModule e InMemoryWebApiModule.

Abrir: app.routes.ts

*Aquí defino rutas como drivers, **driver-detail/:id**, **driver-form/:id**, **cars** y **circuits**. Las rutas con **:id** permiten el detalle y la edición. También activo `scrollPositionRestoration: 'top'`.*

Abrir: driver-detail.component.ts

En este componente, recupero el parámetro con:

```
const id = this.route.snapshot.paramMap.get('id');
```

Abrir: f1-data.service.ts

Aquí uso HttpClient para las APIs: api/pilotos, api/autos, api/circuitos. Uso BehaviorSubject para mantener el estado reactivo. También gestiono errores con handleError<T>().

Ahora es el momento del backend simulado.

Abrir: in-memory-data.service.ts

Este archivo simula el backend con datos reales de pilotos, coches y circuitos.

4:30 – 5:20 | Demostración

Mostrar Terminal: *ng serve en ejecución*

Mostrar Navegador: *abrir app*

Mostrar Navbar: *Navegar a Pilotos*

5:20 – 6:10 | Lista y signals

Abrir en VS Code: *driver-list.component.ts*

Aquí uso Signals y enseñar: *isEditMode, loading, error.*

La lista tiene búsqueda en tiempo real.

En Navegador: *click en un piloto → Detalles*

La URL incluye el ID, y desde aquí puedo editar.

Abrir: driver-form.component.ts

Se construyo el formulario con FormBuilder y valido campos en tiempo real.

6:50 – 7:30 | Resumen

En resumen, este proyecto demuestra:

- *Arquitectura modular*

- *Routing con parámetros*
 - *Formularios reactivos*
 - *Signals*
 - *BehaviorSubjects*
 - *Backend simulado*
 - *Manejo profesional de errores*
-

“Hola, mi proyecto se llama F1 Universe.”

“Es una app Angular 21 para consultar pilotos, monoplazas y circuitos.”

“La parte CRUD está en pilotos: crear, editar y eliminar.”

“Coches y circuitos son de consulta.”

Mostrar la app en navegador, página inicio.

“Voy a enseñar la estructura del proyecto.”

“Tengo componentes separados por responsabilidad.”

“Cada componente tiene HTML, CSS y TypeScript.”

Abrir app

Abrir components

Abrir piloto.interface.ts

“Este es el módulo principal.”

“Aquí declaro los componentes.”

“También importo Router, formularios reactivos, HttpClient e InMemory.”

Abrir app.module.ts

“Aquí están las rutas principales.”

“Uso rutas con parámetro para detalle y edición.”

“También tengo redirección por defecto y restauración de scroll.”

Abrir app.routes.ts

Señalar driver-detail/:id y driver-form/:id.

“En detalle, leo el id desde la URL.”

“Con ese id recupero el piloto correspondiente.”

Abrir driver-detail.component.ts

“Este es el servicio principal de datos.”

“Uso BehaviorSubject para estado global de pilotos.”

“Tengo Signals para loading, error e isEditMode.”

“Manejo errores HTTP de forma centralizada.”

Abrir f1-data.service.ts

Señalar pilotos\$, handleError, y endpoints api/*.

“Este archivo simula el backend con datos reales de F1.”

“Así puedo desarrollar y probar sin servidor externo.”

Abrir [in-memory-data.service.ts](#)

“Ahora demuestro la app funcionando.”

“Navego con el navbar.”

“Voy a la lista de pilotos y aplico filtro.”

“Entro a detalle por ID desde la URL.”

Navegar: Inicio → Pilotos → Detalles de un piloto.

“Aquí construyo el formulario con FormBuilder.”

“Si hay id en ruta, cargo modo edición.”

“Si no hay id, es alta de piloto.”

“Las validaciones se aplican en tiempo real.”

Abrir driver-form.component.ts

Mostrar editar un piloto y guardar.

“En resumen, el proyecto incluye:”

“Arquitectura modular.”

“Routing con parámetros.”

“Formularios reactivos y validaciones.”

“Signals + BehaviorSubject para estado.”

“HttpClient e InMemory Web API.”

“Manejo de errores y flujo CRUD completo en pilotos.”

Pestaña 2

